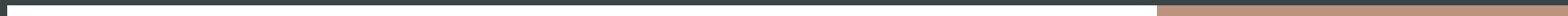


포트폴리오

- 게임 플레이



목차

1. 자체 제작 라이브러리를 이용한
각 종 서버 제작 및 테스트
2. Lock Free 자료구조

프로젝트 개요

- 세션 락 제거를 통한 IOCP 네트워크 라이브러리 구현
- Group 기반 Game Library 구현
- Chat / Login / Game / Monitoring 서버 구성
- 장시간 무중단 테스트를 통한 신뢰성 검증
- Zero Copy On / Off 테스트

NETWORKLIBRARY 구조

라이브러리 구조

- IOCP 기반 비동기 처리
- Send 스레드 (On/Off 가능)
- 세션 배열 및 락프리 스택(Index 관리)
- 세션 Key기반 통신

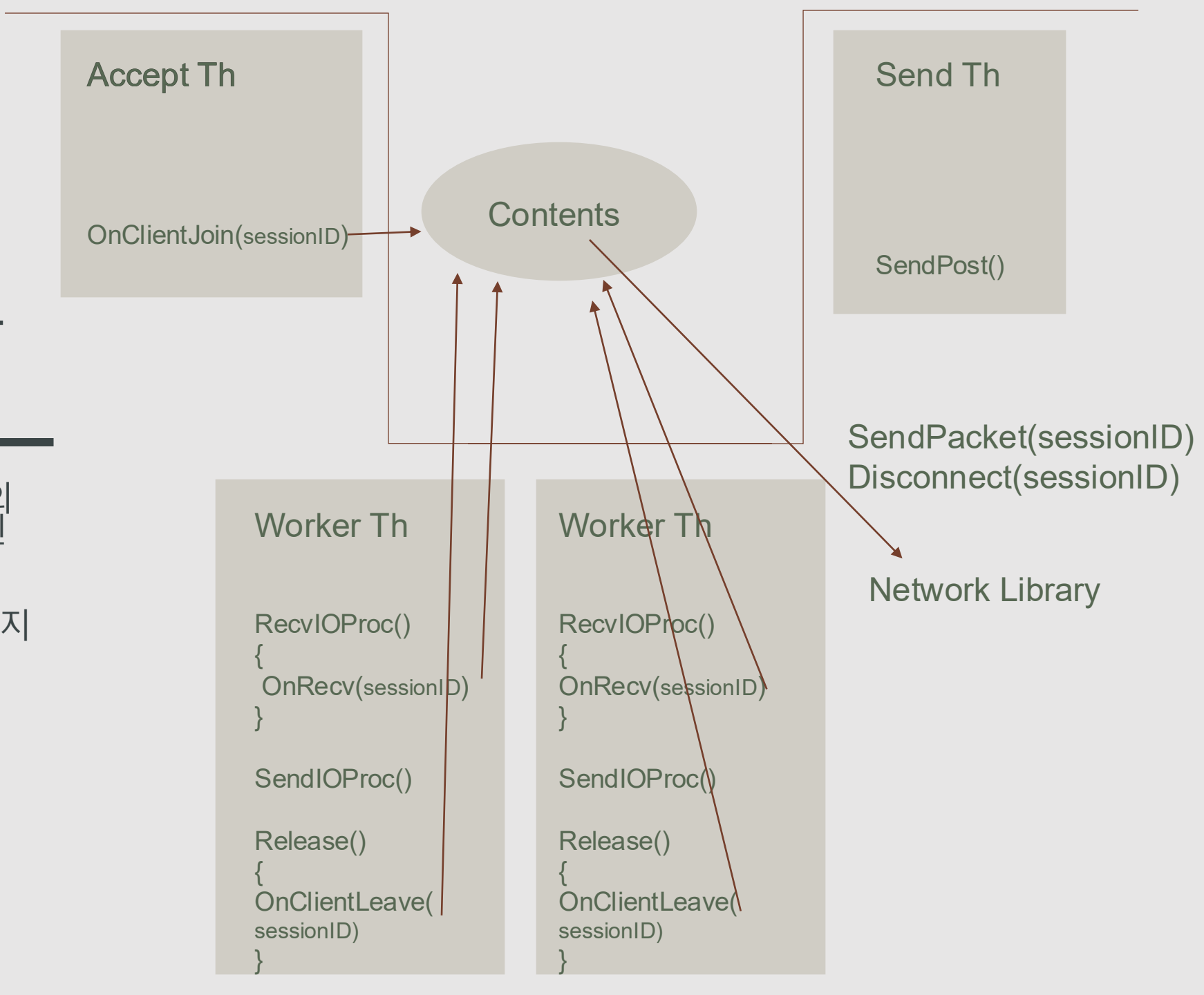
세션 구조

- 송신 큐 : 락프리 큐
- 참조 카운트 기반 세션 유효성
관리

1. 자체 제작 라이브러리를 이용한
각 종 서버 제작 및 테스트

네트워크 라이브러리 스레드 모델

- 콜백 함수를 통해 컨텐츠는 세션의 생성, 메시지 수신, 연결 끊김 확인 가능
- 컨텐츠는 2개의 함수를 통해 메시지 송신 및 연결 끊기를 네트워크 라이브러리에 요청 가능



1. 자체 제작 라이브러리를 이용한

각종 서버 제작 및 테스트

SESSION 관리 코드1

- 세션 구조체 주소 찾기
- 세션 메모리 자체의 유효성 판단

```
bool CNetServer::SendPacket(UINT64 SessionID, CMessage* pMessage)
{
    CSession* pSession;
    DWORD curThreadId = GetCurrentThreadId();

    FindSession(SessionID, &pSession);

    //못 찾은 것임
    if (pSession == nullptr)
        return false;

    // 세션 찾았으면 유효성 확인해야 함.
    if (!SessionInvalid(pSession, SessionID))
    {
        return false;
    }
}
```

```
void CNetServer::FindSession(UINT64 SessionID, CSession** ppSession)
{
    UINT64 index;

    *ppSession = nullptr;

    if (SessionID == df_INVALID_SESSIONID)
        return;

    // 인자로받은 세션 ID에서 Index 추출
    index = SessionID >> INDEX_POS;

    *ppSession = &m_SessionTable[index];
}
```

```
bool CNetServer::SessionInvalid(CSession* pSession, UINT64 SessionID)
{
    InterlockedIncrement64(&pSession->m_RefCnt);

    //그런데 이미 누가 Release 하고 있으면 쓰면 안되니 감소 시키고 Release
    if (pSession->m_RelFlag == 1)
    {
        Release(pSession, pSession->m_SessionID, InterlockedDecrement64((long long*)&pSession->m_RefCnt));
        return false;
    }

    //내가 찾던 세션이 아니라면 증가시킨것 감소
    if (SessionID != pSession->m_SessionID)
    {
        Release(pSession, pSession->m_SessionID, InterlockedDecrement64((long long*)&pSession->m_RefCnt));
        return false;
    }

    return true;
}
```

1. 자체 제작 라이브러리를 이용한 각종 서버 제작 및 테스트

SESSION 관리 코드2

- RELEASE 성공 여부 체크
- 세션 초기화 작업

```
bool CNetServer::Release(CSession* pSession, long retIOCount)
{
    CMessage* peek = nullptr;
    st_IOFLAG check;
    check.Cnt = 0;
    check.Flag = 0;

    if (!InterlockedCompareExchange128(&pSession->m_RefCnt, 1, 0, (long long*)&check))
    {
        return false;
    }

    PostQueuedCompletionStatus(m_IOCP, -1, (ULONG_PTR)pSession, (LPOVERLAPPED)en_RELEASE);

    return true;
}

void CSession::Init(SOCKET socket, UINT64 sessionId)
{
    m_Socket = socket;
    m_SessionID = sessionId;
    m_SendMsgCnt = 0;
    m_RecvQ.Clear();
    m_SendQ.Clear();

    InterlockedIncrement64(&m_RefCnt);
    InterlockedExchange16(&m_SendFlag, 0);
    InterlockedExchange(&m_DCFlag, 0);
    InterlockedExchange64(&m_ReIFlag, 0);
}
```

1. 자체 제작 라이브러리를 이용한 각종 서버 제작 및 테스트

SESSION 관리 코드3

- 실제 RELEASE 처리 과정
- 버퍼 및 INDEX 반납
- 컨텐츠에게 전달

```
void CNetServer::ReleaseProc(CSession* pSession)
{
    CMessage* peek = nullptr;

    /*
     * 실제 Release 작업
     */

    //SendArray에 있는 직렬화 버퍼 반납.(SendPost 내부에서 Release될 수 있음)
    for (int i = 0; i < pSession->m_SendMsgCnt; i++)
    {
        CMessage::Free(pSession->m_SendArray[i]);
    }

    //Send 락프리 큐에 있는 직렬화 버퍼 반납(SendPacket 함수에서 Release 될 수 있음)
    while (1)
    {
        //빠낼게 없으면 탈출
        if (!pSession->m_SendQ.Dequeue(peek))
            break;

        CMessage::Free(peek);
    }

    OnClientLeave(pSession->m_SessionID);

    //그 플래그를 바꾼 스레드가 EmptyIndexStack에 index를 push 해야 함.
    //index 추출
    uint16_t index;

    index = pSession->m_SessionID >> INDEX_POS;
    closesocket(pSession->m_Socket);
    m_pSessionIdxStack->Push(index);
    InterlockedDecrement16(&m_CurSessionCnt);
}
```

IOCP 통한 IO 처리

- RECV, SEND IO 처리
및 RELEASE 처리
- 워커 스레드 종료 판단

```
while (!ESC)
{
    DWORD      cbTransferred = 0;
    OVERLAPPED* pOverlapped = nullptr;    // IO에 사용된 Overlapped 구조체 주소값 or Task Type
    CSession*   pSession = nullptr;

    retval = GetQueuedCompletionStatus(m_IOCP, &cbTransferred, (PULONG_PTR)&pSession, (LPOVERLAPPED*)&pOverlapped, INFINITE);

    //false 인 상황(IOCP 완료 통지 큐에서 디큐잉이 안된 경우, IOCP 핸들이 CloseHandle로 인해 닫힌 경우)
    if (retval == false)
    {
        //IOCP가 닫힐 때
        if (pOverlapped == nullptr)
        {
            err = GetLastError();
            LOG(L"CNetLibrary", en_LOG_LEVEL::dfLOG_LEVEL_SYSTEM, L"WorkerThread GQCS IOCP Failed / Error Code : %d", err);

            ESC = true;
            break;
        }

        //WSASend, WSAREcv로 요청한 IO가 실패한 경우(TCP가 끊긴 것임)
        else
        {
            err = GetLastError();
            LOG(L"CNetLibrary", en_LOG_LEVEL::dfLOG_LEVEL_DEBUG, L"WorkerThread GQCS IO Failed / Error Code : %d", err);
        }
    }

    if (pOverlapped == (LPOVERLAPPED)en_RELEASE)
    {
        ReleaseProc(pSession);
    }

    else if (pOverlapped == &pSession->m_RecvOverlapped)
    {
        RecvIOProc(pSession, cbTransferred);
    }

    else if (pOverlapped == &pSession->m_SendOverlapped)
    {
        SendIOProc(pSession, cbTransferred);
    }
}
```

GAMELIBRARY 구조

그룹 로직 처리

- 그룹 로직 단일 스레드 처리(Lock 이용)
- 그룹간 이동시 PQCS 사용
- 컨텐츠가 그룹 상속, 유저 객체는 IUser 상속
- 프레임 스레드에서 각 그룹 프레임 로직 PQCS

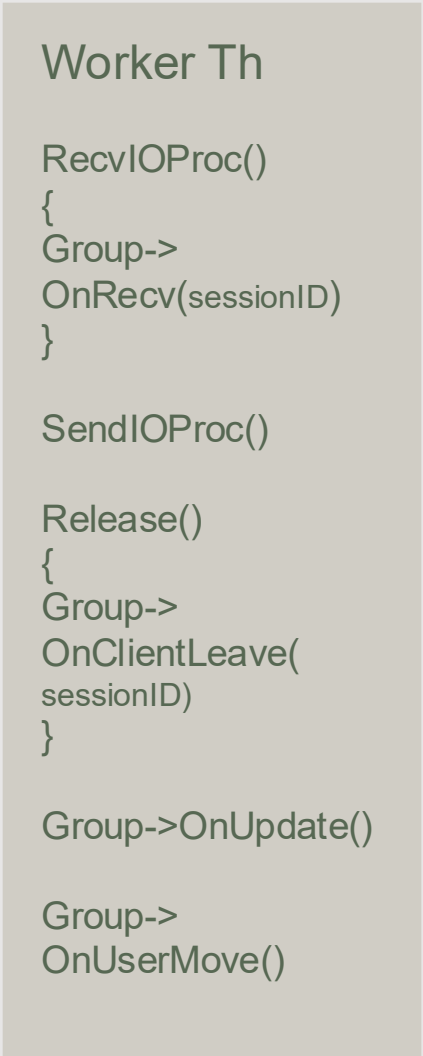
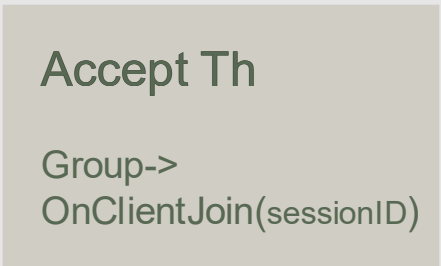
네트워크 로직 처리

- 기존 네트워크 라이브러리 동작과 거의 동일
- 세션에 GroupID 멤버 적용

1. 자체 제작 라이브러리를 이용한
각 종 서버 제작 및 테스트

게임 라이브러리 스레드 모델

- 세션 멤버로 그룹 ID 존재
- 그룹 클래스의 콜백 함수 호출
- 프레임 스레드에서 PQCS



1. 자체 제작 라이브러리를 이용한 각종 서버 제작 및 테스트

그룹 클래스

```
class CGroup
{
private:
    //////////////////////////////////////
    // 그룹 객체 변수
    //////////////////////////////////////
    CGameLibrary*      m_pGameLib;           // 게임 라이브러리 네트워크 및 그룹 등 함수 사용하기 위한 멤버
    SRWLOCK             m_GroupLock;         // 그룹을 상속 받는 콘텐츠 직렬 처리를 위한 Lock
    DWORD              m_GroupID;           // 그룹을 상속 받는 콘텐츠의 실제 타입 구분자
    UINT               m_GroupFrameTime;    // 프레임 로직 돌릴때 프레임(ms 단위)
    DWORD              m_OldTime;           // 프레임 스레드에서 체크할 시간
    BOOL               m_Shared;            // 멀티 스레드가 그룹에 접근 가능해도 되는지 제어 플래그

    //////////////////////////////////////
    // 콘텐츠가 구현할 Callback 함수
    //////////////////////////////////////
    virtual void OnClientJoin(UINT64 sessionId) = 0;
    virtual void OnClientLeave(UINT64 sessionId) = 0;
    virtual void OnRecv(UINT64 sessionId, CMessage* pMessage) = 0;
    virtual void OnUserMove(UINT64 sessionId, IUser* pUser) = 0;
    virtual void OnUpdate() = 0;

    //////////////////////////////////////
    // 게임 라이브러리 클래스 제공할수 래핑
    //////////////////////////////////////
    inline bool SendPacket(UINT64 SessionID, CMessage* pMessage)
    {
        return m_pGameLib->SendPacket(SessionID, pMessage);
    }
    inline bool Disconnect(UINT64 SessionID)
    {
        return m_pGameLib->Disconnect(SessionID);
    }
    inline bool FindIP(UINT64 SessionID, std::wstring& OutIP)
    {
        return m_pGameLib->FindIP(SessionID, OutIP);
    }

    inline bool GroupMove(std::wstring ToContents, UINT64 sessionId, IUser* pUser)
    {
        return m_pGameLib->GroupMove(ToContents, sessionId, pUser);
    }
}
```

1. 자체 제작 라이브러리를 이용한 각 종 서버 제작 및 테스트

그룹 처리 코드1

```
bool CGameLibrary::Attach(CGroup* pContents, wstring contentsType, UINT64 groupframe, BOOL shared)
{
    m_GroupArray.push_back(pContents);
    pContents->SetGroupFrame(groupframe);
    pContents->SetGroupID(m_GroupID);
    pContents->SetGameLib(this);
    pContents->SetSharedFlag(shared);
    m_GroupMap.insert(std::pair<std::wstring, UINT16>(contentsType, m_GroupID));
    m_GroupID++;

    return true;
}

void CGameLibrary::FrameThread()
{
    DWORD curTime;
    DWORD oldTime;
    INT cnt;
    UINT64 frametime;

    LOG(L"GameLibrary", en_LOG_LEVEL::dfLOG_LEVEL_SYSTEM, L"FrameThread Start : %d ", GetCurrentThreadId());
    while (!m_Endflag)
    {
        for (int i = 0; i < m_GroupID; i++)
        {
            oldTime = m_GroupArray[i]->GetOldTime();
            curTime = timeGetTime();
            frametime = m_GroupArray[i]->GetGroupFrame();
            cnt = (curTime - oldTime) / frametime;

            for (int j = 0; j < cnt; j++)
            {
                PostQueuedCompletionStatus(m_IOCP, NULL, (ULONG_PTR)i, (LPOVERLAPPED)en_FRAME);
                m_GroupArray[i]->SetOldTime();
            }
        }
    }

    LOG(L"GameLibrary", en_LOG_LEVEL::dfLOG_LEVEL_SYSTEM, L"FrameThread End... : %d ", GetCurrentThreadId());
    return;
}
```

1. 자체 제작 라이브러리를 이용한 각 종 서버 제작 및 테스트

그룹 처리 코드2

```
void CGameLibrary::FrameProc(UINT16 groupId)
{
    m_GroupArray[groupId]->ExclusiveGroupLock();
    m_GroupArray[groupId]->OnUpdate();
    m_GroupArray[groupId]->ExclusiveGroupUnlock();
}
```

```
UINT16 id = pSession->m_GroupID;
if (m_GroupArray[id]->GetSharedFlag())
{
    m_GroupArray[id]->SharedGroupLock();
    m_GroupArray[id]->OnRecv(pSession->m_SessionID, pPacket);
    m_GroupArray[id]->SharedGroupUnlock();
}
else
{
    m_GroupArray[id]->ExclusiveGroupLock();
    m_GroupArray[id]->OnRecv(pSession->m_SessionID, pPacket);
    m_GroupArray[id]->ExclusiveGroupUnlock();
}
```

```
void CGameLibrary::GroupMoveProc(CMessage* pMessage)
{
    UINT64 sessionId;
    IUser* pUser;
    CSession* pSession;
    UINT16 groupId;

    *pMessage >> sessionId;
    *pMessage >> (UINT64%)pUser;
    *pMessage >> groupId;
    CMessage::Free(pMessage);

    FindSession(sessionID, &pSession);
    if (pSession == nullptr)
    {
        __debugbreak();
    }

    m_GroupArray[groupId]->ExclusiveGroupLock();
    m_GroupArray[groupId]->OnUserMove(sessionID, pUser);
    m_GroupArray[groupId]->ExclusiveGroupUnlock();

    pSession->m_GroupID = groupId;

    Release(pSession, InterlockedDecrement64(&pSession->m_RefCnt));
}
```

서버 모듈 구성

채팅 서버

- 싱글 스레드 구조
- 인증 처리 (Redis 서버 통신)

로그인 서버

- DB 통신(= 플랫폼 통신)
- Redis 서버 토큰 저장

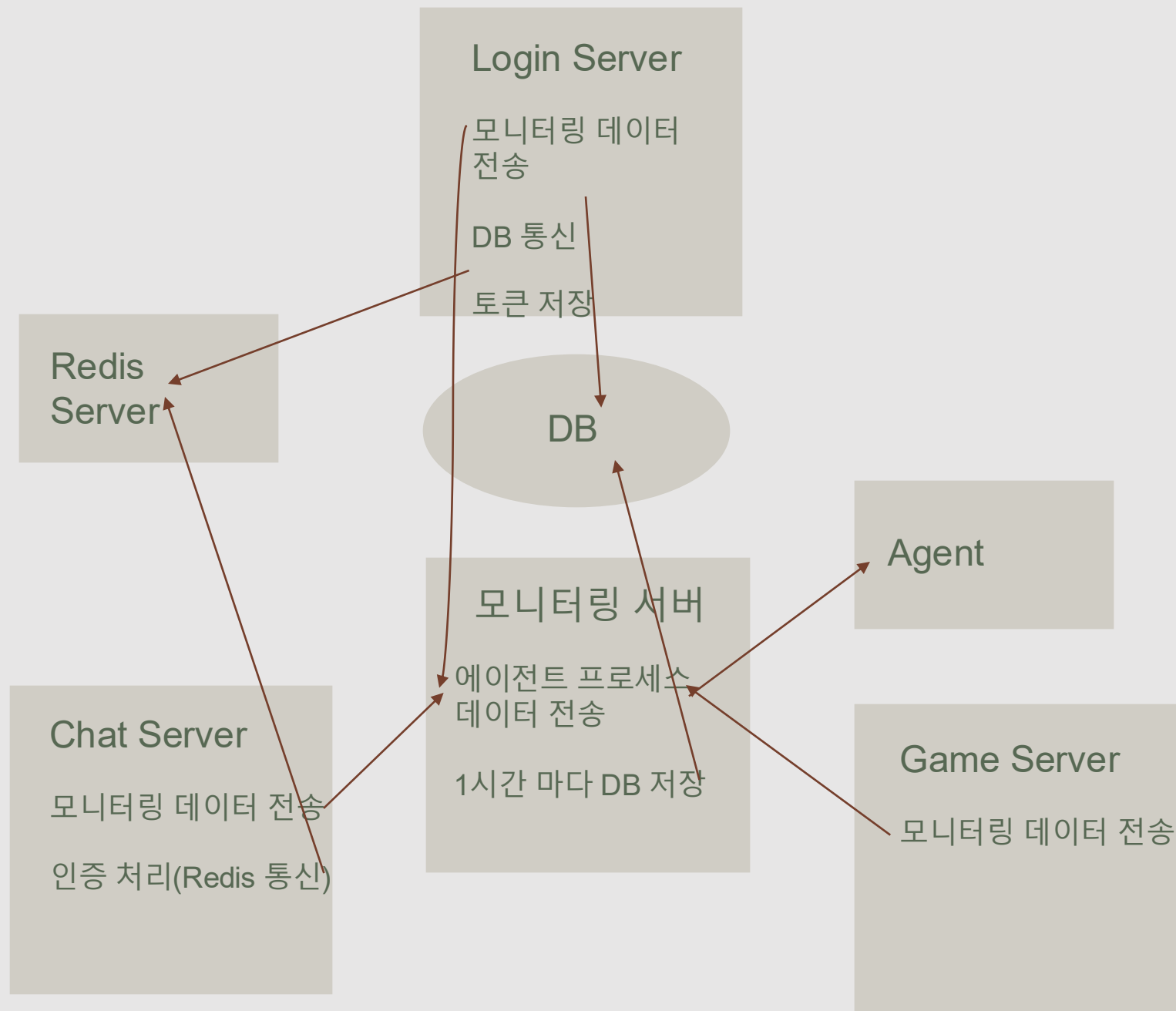
게임 서버

- 인증 콘텐츠(유저 객체 생성)
- 에코 콘텐츠(로그인 응답 메시지 송신)

모니터링 서버

- 각 서버의 모니터링 클라이언트, 모니터링 데이터 전송
- 전송 받은 모니터링 데이터 에이전트 프로세스 전송
- 1시간 마다 모니터링 데이터 DB 저장

서버 전체 구조



1. 자체 제작 라이브러리를 이용한 각종 서버 제작 및 테스트

채팅 서버 인증 처리 1

```
void ChatServer::LoginProc(CMessage* pMessage, UINT64 sessionid)
{
    if (m_UserMap.size() > m_UserMaxCnt)
    {
        LOG(L"ChatServer", en_LOG_LEVEL::dfLOG_LEVEL_ERROR, L"LoginRequest::UserMax / UniqID : %llu ", sessionid);
        Disconnect(sessionid);
    }

    return;
}

AuthRequestJob(sessionid, pMessage);
}

void ChatServer::AuthThread()
{
    JOB* job;
    BOOL esc = false;

    LOG(L"ChatServer", en_LOG_LEVEL::dfLOG_LEVEL_SYSTEM, L"AuthThread Start... : %d ", GetCurrentThreadId());

    while (!esc)
    {
        WaitForSingleObject(m_AuthEvent, INFINITE);

        while (m_pAuthJobQ->GetUseSize() > 0)
        {
            //Deq하는 스레드가 1개라서 size가 0보다 크면 Enq 스레드에서 큐에 노드를 넣었는데 없을 수 없음.
            if (!m_pAuthJobQ->Dequeue(job))
                __debugbreak();

            switch (job->s_Type)
            {
            case en_AuthReq:
                AuthReqProc(job->s_Id, job->s_ptr);
                break;

            case en_End:
                esc = true;
                break;
            }

            m_pJobPool->Free(job);
            m_AuthTPS++;
        }
    }

    LOG(L"ChatServer", en_LOG_LEVEL::dfLOG_LEVEL_SYSTEM, L"AuthThread End... : %d ", GetCurrentThreadId());
}
```

1. 자체 제작 라이브러리를 이용한 각종 서버 제작 및 테스트

채팅 서버 인증 처리2

```
void ChatServer::AuthReqProc(UINT64 sessionId, CMessage* pMessage)
{
    INT64 accountNo = *reinterpret_cast<INT64*>(pMessage->GetReadPos());

    m_pRedisClient->get(to_string(accountNo), [this, pMessage, sessionId](cpp_redis::reply& reply)
    {
        INT64 accountNo = *reinterpret_cast<INT64*>(pMessage->GetReadPos());

        // 토큰 자체가 없으면 해당 유저 연결 끊기
        if (reply.is_null()) {
            LOG(L"ChatServer", en_LOG_LEVEL::dfLOG_LEVEL_ERROR, L"AuthProc No Token ... / UniqID : %lld / AccountNo : %lld ", sessionId,
                CMessage::Free(pMessage);
                Disconnect(sessionID);
                return;
            }

            std::string value = reply.as_string();
            std::string token(pMessage->GetReadPos() + sizeof(INT64) + ID_MAX * sizeof(WCHAR) + NICK_MAX * sizeof(WCHAR), SESSION_KEY_MAX);

            // 토큰이 있으면 인증 완료 되었으니 Update 스레드에게 후처리 전달
            if (value == token)
            {
                AuthCompleteJob(sessionID, pMessage);
            }

            // 토큰이 다르면 해당 세션 연결 끊기
            else
            {
                LOG(L"ChatServer", en_LOG_LEVEL::dfLOG_LEVEL_ERROR, L"AuthProc TokenInvalid ... / UniqID : %lld / AccountNo : %lld / RedisToken : %s",
                    CMessage::Free(pMessage);
                    Disconnect(sessionID);
            }

            // Redis에 토큰 삭제 (비동기 방식)
            std::vector<std::string> delvec;
            delvec.push_back(to_string(accountNo));

            m_pRedisClient->del(delvec, [])(cpp_redis::reply& del_reply) {
                });

            m_pRedisClient->commit();
            return;
        });

    m_pRedisClient->commit();
}
```

1. 자체 제작 라이브러리를 이용한 각종 서버 제작 및 테스트

로그인 서버 로그인 처리

```
// 세션의 IP 얻고 유저 객체에 저장
if (!FindIP(sessionid, IP))
{
    Disconnect(sessionid);
    return;
}

FindServerInfo(IP, &OutInfo);

// 현재는 더미는 사실 ip 대역이라서 만약 저장되지 않는 다른 대역으로 세션의 ip 대역
// 원래는 더미와 실제 클라가 로그인 서버에 접속 가능한 환경이면 문제가 없는데 현재
if (OutInfo == nullptr)
{
    Disconnect(sessionid);
    return;
}

// DB 통신
GetDBData(ID, NICK, AccountNo);

// 토큰 서버에 토큰 저장(플랫폼이 발급한 세션key를 클라이언트가 들고왔고 로그인 서
std::string token(SessionKey, SESSION_KEY_MAX);
SetRedisToken(AccountNo, token);

// 유저 객체 초기화
pUser = m_pUserPool->Alloc();
pUser->User_Init(sessionid, AccountNo);

// 유저 자료구조에 넣기
UserInsert(sessionid, pUser);

Status = dfLOGIN_STATUS_OK;

//로그인 서버 로그인 응답 패킷 생성 및 전송
CMessage* pPacket = CMessage::Alloc();
pPacket->Clear();

*pPacket << (WORD)en_PACKET_CS_LOGIN_RES_LOGIN;
*pPacket << AccountNo;
*pPacket << Status;
pPacket->PutData((CHAR*)ID, ID_MAX * sizeof(WCHAR));
pPacket->PutData((CHAR*)NICK, NICK_MAX * sizeof(WCHAR));
pPacket->PutData((CHAR*)OutInfo->s_ServerIP.c_str(), IP_LEN * sizeof(WCHAR));
*pPacket << OutInfo->s_GamePort;
pPacket->PutData((CHAR*)OutInfo->s_ServerIP.c_str(), IP_LEN * sizeof(WCHAR));
*pPacket << OutInfo->s_ChatPort;

SendPacket(sessionid, pPacket);

CMessage::Free(pPacket);

endTime = timeGetTime();
```

1. 자체 제작 라이브러리를 이용한 각종 서버 제작 및 테스트

게임 서버 컨텐츠 처리

```
void CAuth::LoginRequetProc(UINT64 sessionID, CMessage* pMessage)
{
    CUser* pUser = nullptr;
    INT64 accountNo;
    CHAR sessionkey[df_SESSIONKEY_MAX];
    int version;

    *pMessage >> accountNo;
    pMessage->GetData(sessionkey, df_SESSIONKEY_MAX);
    *pMessage >> version;

    if (pMessage->GetLastError())
    {
        LOG(L"Auth", en_LOG_LEVEL::dfLOG_LEVEL_DEBUG, L"EchoRequest::CM
        //프로토콜 보다 보낸 데이터 크기가 적으면 플래그 꺼짐.
        Disconnect(sessionID);
        return;
    }

    if (pMessage->GetDataSize() > 0)
    {
        LOG(L"Auth", en_LOG_LEVEL::dfLOG_LEVEL_DEBUG, L"EchoRequest::CM
        //프로토콜 보다 보낸 데이터 크기가 크면 끊기
        Disconnect(sessionID);
        return;
    }

    // NonUser 자료구조에서 제거
    m_NonUserMap.erase(sessionID);

    // 유저 객체 생성
    pUser = CUser::UserAlloc();
    pUser->Clear(sessionID, accountNo);

    // 그룹 이동
    if (!GroupMove(L"Echo", sessionID, dynamic_cast<IUser*>(pUser)))
    {
        // 그룹 이동 실패시(그 사이에 세션이 무효화 된 경우)
        CUser::UserFree(pUser);
    }

    return;
}
```

```
void CEcho::EchoRequestProc(UINT64 sessionID, CMessage* pMessage)
{
    INT64 accountNo;
    LONGLONG tick;
    CUser* pUser = nullptr;

    // 메세지 받고 에러 체크

    *pMessage >> accountNo;
    *pMessage >> tick;

    if (pMessage->GetLastError())
    {
        LOG(L"Echo", en_LOG_LEVEL::dfLOG_LEVEL_DEBUG, L"EchoRequest::CMessag
        //프로토콜 보다 보낸 데이터 크기가 적으면 플래그 꺼짐.
        Disconnect(sessionID);
        return;
    }

    if (pMessage->GetDataSize() > 0)
    {
        LOG(L"Echo", en_LOG_LEVEL::dfLOG_LEVEL_DEBUG, L"EchoRequest::CMessag
        //프로토콜 보다 보낸 데이터 크기가 크면 끊기
        Disconnect(sessionID);
        return;
    }

    // 에코 응답 생성
    std::unordered_map<UINT64, CUser*>::iterator it = m_EchoUser.find(sessionID);
    if (it == m_EchoUser.end())
        __debugbreak();

    pUser = it->second;
    pUser->m_recvTime = timeGetTime();

    CMessage* pPacket = CMessage::Alloc();
    pPacket->Clear();

    *pPacket << (WORD)en_PACKET_CS_GAME_RES_ECHO;
    *pPacket << pUser->m_accountNo;
    *pPacket << tick;

    SendPacket(sessionID, pPacket);

    CMessage::Free(pPacket);

    IncSendTPS();
}
```

안정성 검증

채팅 서버 테스트

- 세션 5천개
- 메모리 사용률 112MB 유지
- Update TPS 14000 유지
- RTT 평균 76ms 유지
- 크래시 발생 x, 세션 및 유저 잔존 x, 직렬화 버퍼 잔존 x

로그인 서버 및 채팅 서버 연동 테스트

- 인증 처리 안정성 확인
- Update TPS 5000 이상 유지
- RTT 평균 43ms 유지
- 크래시 발생 x, 세션 및 유저 잔존 x, 직렬화 버퍼 잔존 x

게임 서버

- 에코 메시지 TPS 50만 유지
- 크래시 발생 x, 직렬화 버퍼 잔존x

ZERO COPY TEST

Zero Copy 테스트 환경

- 클라이언트 30개, 서버 / 클라이언트 워커 스레드 3개, Send 스레드
- 서버 세션 마다 보낼 메시지 버퍼 멤버로 등록
- 클라이언트 멤버로 받을 버퍼 등록
- 서버 WSASend Return Time, GQCS TPS 측정
- 클라이언트 단순 WSAREcv 반복

ZERO COPY TEST

Zero Copy On	HighDataSize	LowDataSize
GQCS TPS	75,656	183,912
WSASend Average Return Time(micro sec)	12.77	5.02
Process CPU Total Usage (%)	10.95	13.57
Process CPU User Usage (%)	0.84	2.00
Process CPU Kernel Usage (%)	10.11	11.57

Zero Copy Off	HighDataSize	LowDataSize
GQCS TPS	69,012	206,072
WSASend Return Time	14.03	4.45
Process CPU Total Usage (%)	10.41	13.59
Process CPU User Usage (%)	0.80	2.19
Process CPU Kernel Usage (%)	9.61	11.40

- DataSize 작을 때 Copy Off > On
- Copy cost vs IRQ Create, Destroy, Page Lock Unlock cost

프로젝트 개요

- 멀티 스레드 환경에서 발생하는 실제 문제 재현
- 해결 방법 검증하여 안정적인 동작 확인
- 다른 동기화 방식과 성능 비교 및 고찰

주요 이슈

1. ABA
2. PAGE DECOMMIT
3. 2ND CAS FAIL
4. ORDER REVERSAL
5. SHARED POOL

ABA 원인 및 해결방법

1. 원인

- 메모리 제한적이라 재활용 해야함.
- CAS로 멤버 변수 가리키는 노드 재활용 여부 구분 못함

2. 해결방법

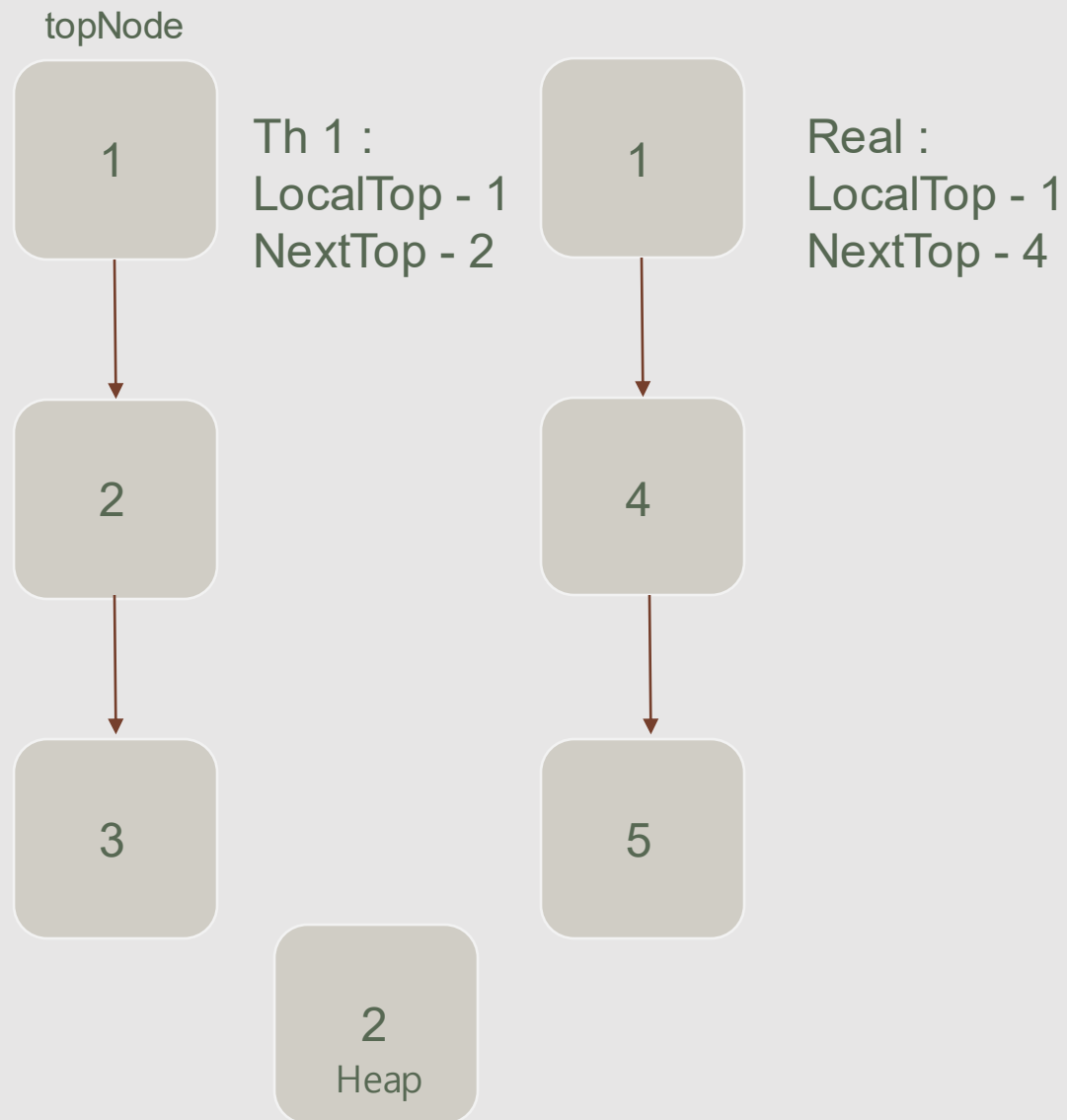
- 멤버 변수가 가리키는 노드 재활용 된 것임을 할 수 있게 하기
- DOUBLE CAS, TAGGING

문제 재현

- <https://youtu.be/tbD-PNxHf5s>

문제 해결

- <https://youtu.be/LjmOqV8cS9M>



PAGE DECOMMIT 원인 및 해결방법

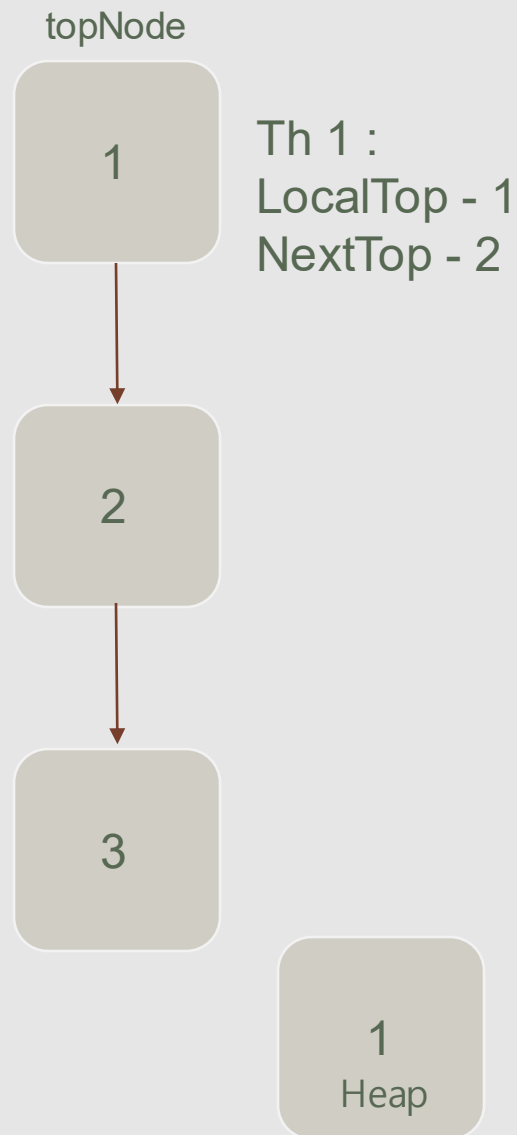
1. 원인
 - 힙이 반납한 노드가 있던 페이지를 힙이 DECOMMIT 시킴
2. 해결방법
 - 메모리 관리 주체를 사용자가 하기
 - 메모리 풀 사용

문제 재현

- <https://youtu.be/tFWJiXKYj5Y>

문제 해결

- <https://youtu.be/pd9q8vR92S4>



2ND CAS FAIL 원인 및 해결방법

1. 원인

- 1ST CAS는 TAIL이 가리키는 NEXT 멤버변수가 NULLPTR인지를 체크하지 LOCALTAIL이 실제 TAIL임을 보장하지 않음.

2. 해결방법

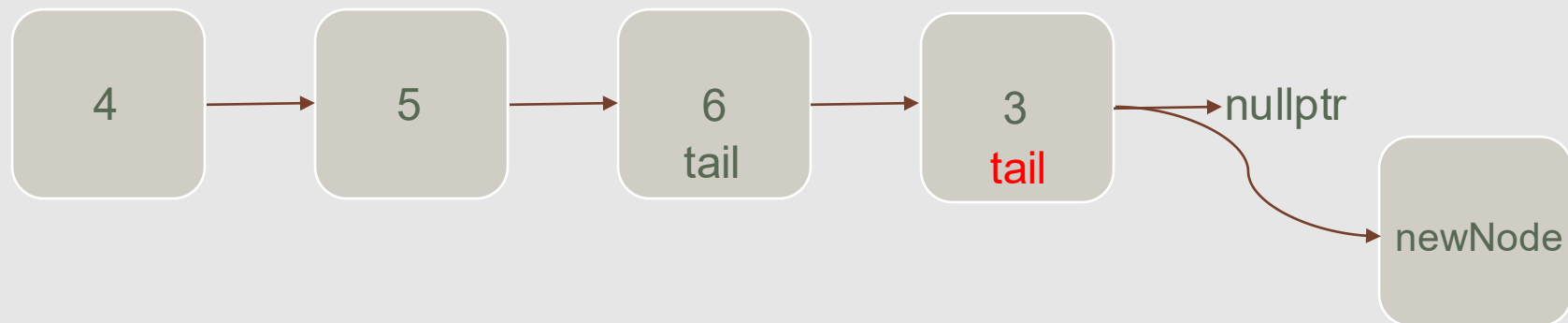
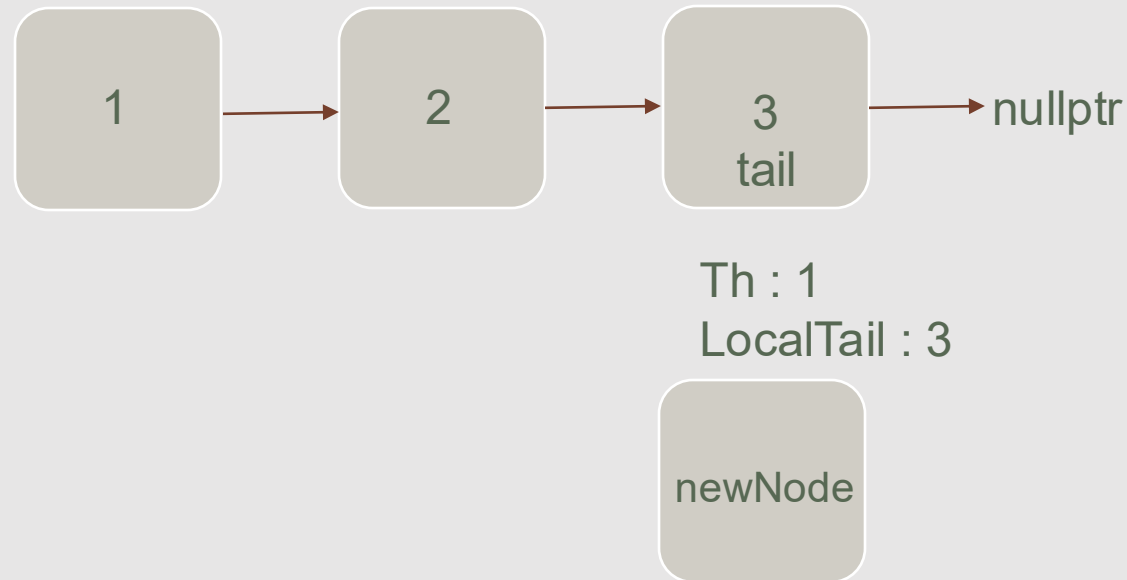
- 2ND CAS 실패 결과 보정.
- ENQ, DEQ 진입시 TAIL 밀어주기

문제 재현

- <https://youtu.be/tFWJiXKYj5Y>

문제 해결

- <https://youtu.be/pd9q8vR92S4>



ORDER REVERSAL 원인 및 해결방법

1. 원인

- 1ST CAS는 TAIL이 가리키는 NEXT 멤버변수가 NULLPTR인지를 체크하지 LOCALTAIL이 실제 TAIL임을 보장하지 않음.

2. 해결방법

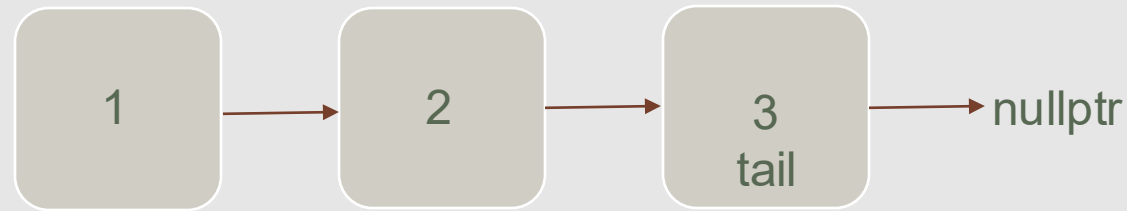
- NEXT를 NULLPTR이 아닌 특정 값으로 초기화

문제 재현

- <https://youtu.be/gUviydSV0wA>

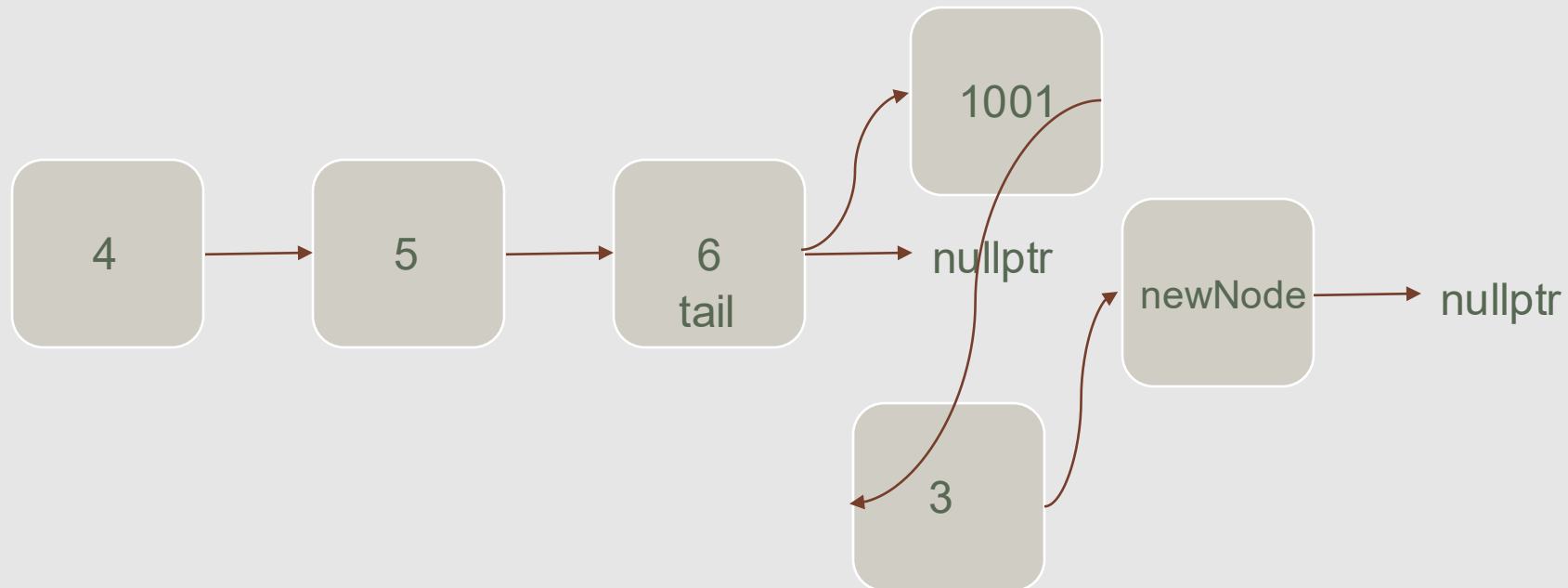
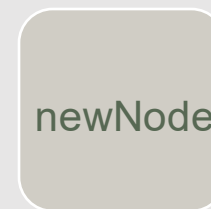
문제 해결

- <https://youtu.be/7gOG7qrjBoM>



Th : 1

LocalTail : 3



SHARED POOL 원인 및 해결방법

1. 원인

- 1ST CAS는 TAIL이 가리키는 NEXT 멤버변수가 NULLPTR인지를 체크하지 LOCALTAIL이 실제 기존 큐의 TAIL임을 보장하지 않음.

2. 해결방법

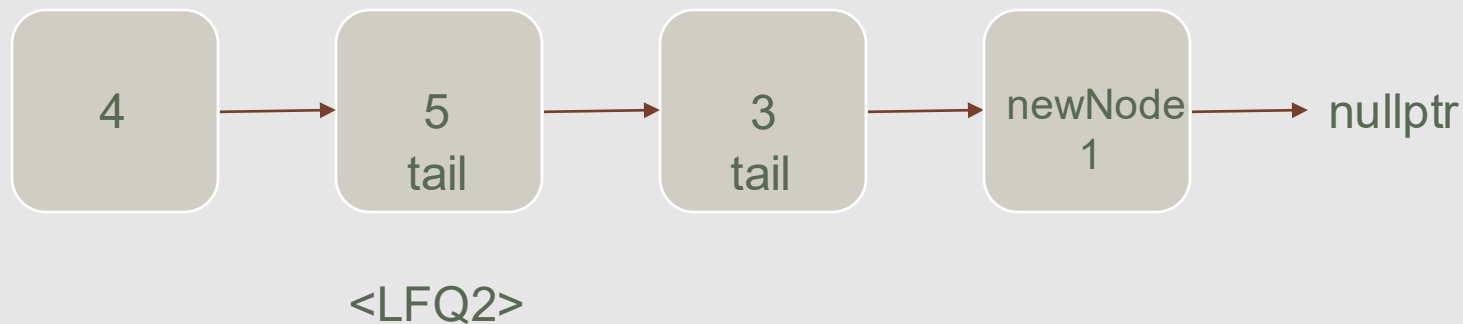
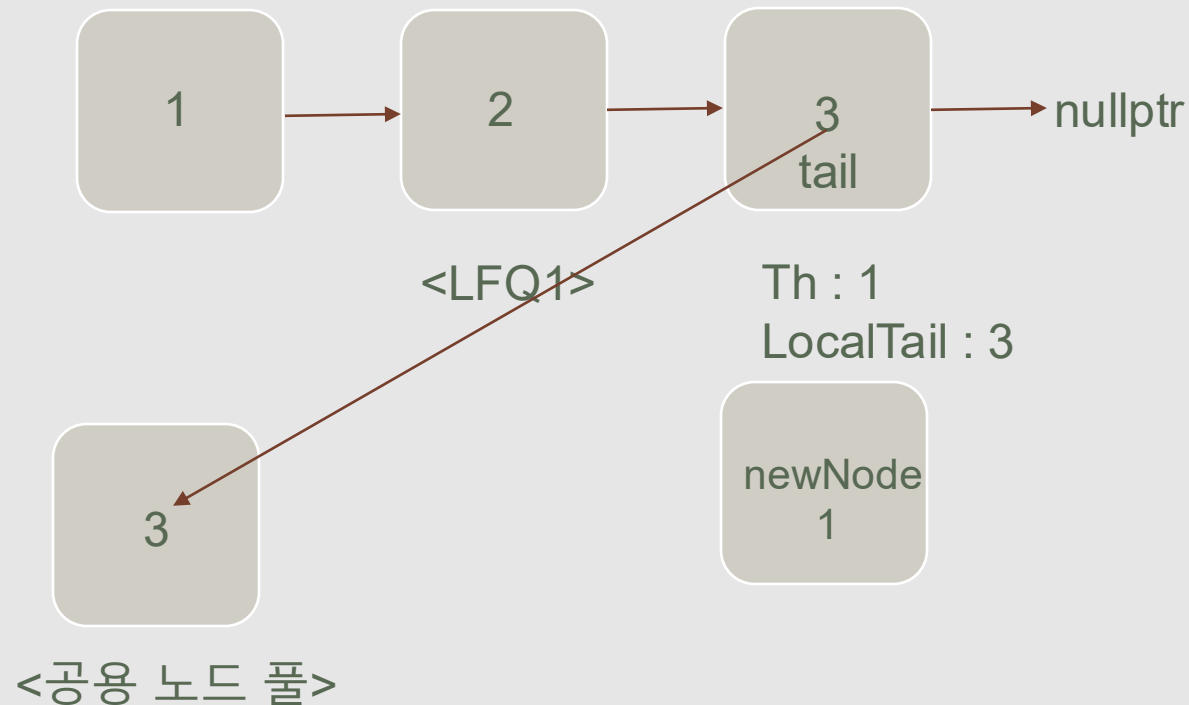
- 1ST CAS 비교시 큐의 ID를 넣어 추가로 비교 하기

문제 재현

- <https://youtu.be/iPoZmHwS0OM>

문제 해결

- <https://youtu.be/Jkn-RgcsvVc>



성능 비교

1. 테스트 환경 및 목적
2. 성능 비교 결과
3. 결과 및 분석

테스트 환경 및 목적

- 목적 : 스레드 수 변화에 따라 SPINLOCK, SRWLOCK, LOCKFREE 성능 비교
- 방법 : 10분동안 스레드 마다 PUSH/POP 시도 후 카운트를 올려 총합 비교

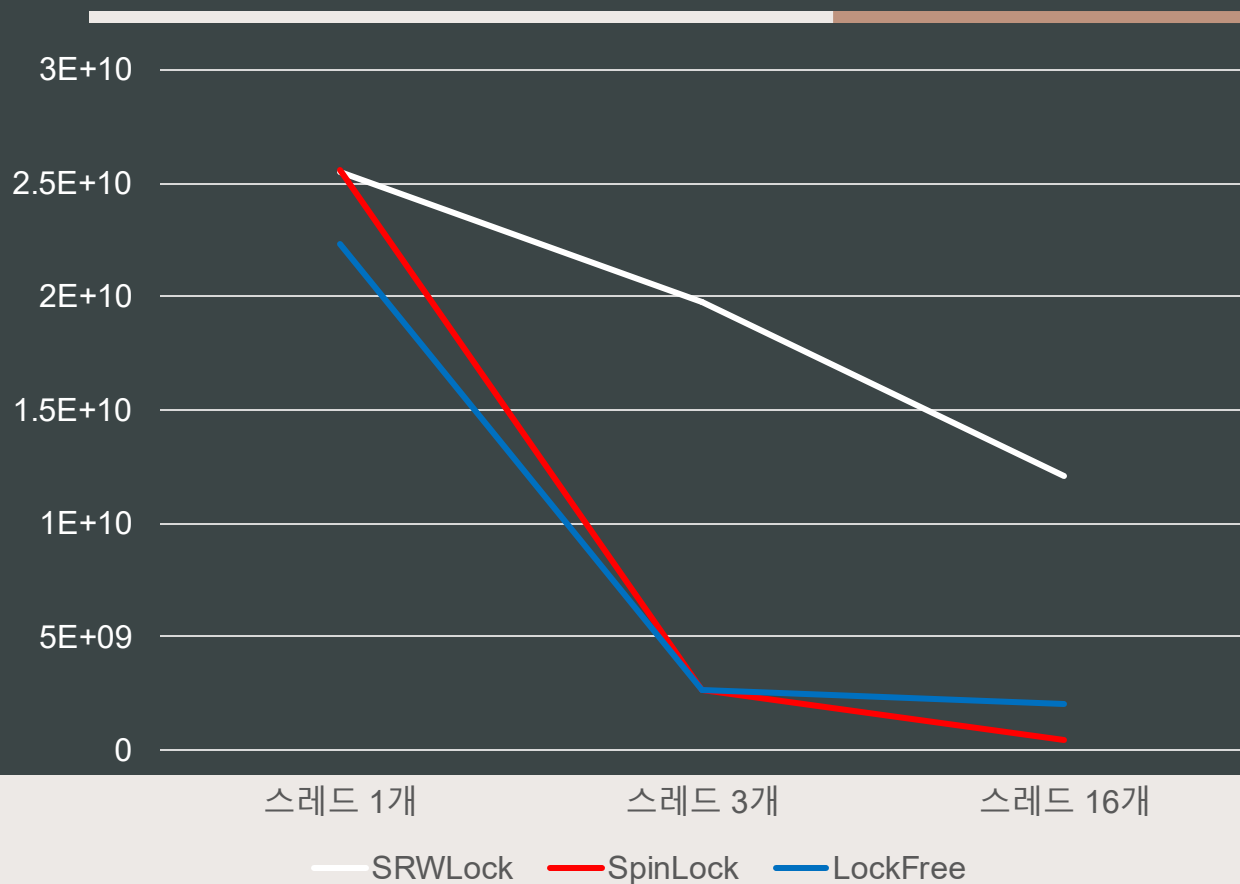
테스트 환경

- Window 10
- Ryzen5 5600x(6코어 12스레드)
- 16GB RAM
- 동일 스택 자료구조

네트워크 로직 처리

- 기존 네트워크 라이브러리 동작과 거의 동일
- 세션에 GroupID 멤버 적용

성능 비교 결과



- 단일 스레드
 - SRWLock = SpinLock > LockFree
- 스레드 3개
 - SRWLock > LockFree > SpinLock
- 스레드 16개
 - SRWLock > LockFree > SpinLock

결과 및 분석

- 단일 스레드 비교

	SRWLock	SpinLock	LockFree
Thread Count	25,476,490,501	25,589,938,990	22,346,230,414
Process CPU Total Usage (%)	8.38	8.39	8.39
Process CPU User Usage (%)	8.33	8.34	8.34
Process CPU Kernel Usage (%)	0.05	0.05	0.05
Process Context Switches (cnt /sec)	5.01	2.15	2.53

- 단일 스레드에서는 경합 발생 없음으로 SpinLock = SRWLock
- CAS > InterlockExchange에 의한 LockFree 성능 감소

결과 및 분석

- 스레드 3개 비교

	SRWLock	SpinLock	LockFree
Thread Count (노드 할당 분리 전)	19,776,245,124	2,631,708,160	2,659,933,145
Thread Count (노드 할당 분리 후)	X	2,795,043,412	X
Process CPU Total Usage (%)	25.04	25.06	25.06
Process CPU User Usage (%)	24.98	25.00	25.00
Process CPU Kernel Usage (%)	0.06	0.06	0.06
Process Context Switches (cnt /sec)	32.03	29.37	28.33
Interlock Try Total Count	X	21,986,291,111	6,271,727,052
Interlock Success Ratio(%)	X	9.82	73.29
Interlock Failed Ratio(%)	X	90.17	26.70
RDTSC (tick / loop)	350.75	2171.40	X

결과 및 분석

- 스레드 16개 비교

	SRWLock	SpinLock	LockFree
Thread Count (노드 할당 분리 전)	12,108,320,327	464,494,023	2,018,193,425
Thread Count (노드 할당 분리 후)	X	716,903,745	X
Process CPU Total Usage (%)	99.38	99.45	99.38
Process CPU User Usage (%)	99.18	99.27	99.19
Process CPU Kernel Usage (%)	0.20	0.18	0.19
Process Context Switches (cnt /sec)	2488.83	1960.63	1989.76
Interlock Try Total Count	X	42,981,774,386	17,095,861,561
Interlock Success Ratio(%)	X	1.18	20.75
Interlock Failed Ratio(%)	X	98.82	79.25

결과 및 분석

- 스레드 3개 비교

- 스레드 3개 일 때 SRLock > LockFree > SpinLock
- SRWLock과 busy-wait 동기화 방식의 차이 : 경합 발생 시 단위 시간당 lock 시행 횟수에 따른 명령어 부하
- SpinLock vs LockFree 차이 : 노드 생성 및 초기화 병렬 처리, interlock 명령어 부하 차이, interlock 명령어 시행 횟수 차이

결과 및 분석

- 스레드 3개 비교

- SRWLOCK의 어셈블리임.
- 처음에는 LOCK 명령어로 소유권 확보를 시도함
- 실패 하면 어떤 로직을 처리하다가 BACKOFF 함수를 호출함.
- 여기 내부에서 그냥 아무런 의미 없이 SPIN을 돌다가 나와서 LOCK CMPXCHG로 소유권 확보를 시도함.

```

00007FFDB3CD90A0  push    rbp
00007FFDB3CD90A2  push    rsi
00007FFDB3CD90A3  sub     rsp,58h
00007FFDB3CD90A7  xor     ebp,ebp
00007FFDB3CD90A9  mov     rsi,rcx
00007FFDB3CD90AC  mov     dword ptr [rsp+70h], ebp
00007FFDB3CD90B0  lock    bts qword ptr [rcx], 0
00007FFDB3CD90B6  jnb     RtlAcquireSRWLockExclusive+20h (07FFDB3CD900Ch)
00007FFDB3CD90BE  add     rsp,58h
00007FFDB3CD90C0  pop     rsi
00007FFDB3CD90C1  pop     rbp
00007FFDB3CD90C2  ret

```

```

00007FFDB3CD9139  mov     dword ptr [rsp+40h], eax
00007FFDB3CD913D  cmovle  rdx, r14
00007FFDB3CD9141  mov     rdx, rbx
00007FFDB3CD9144  test    eax, eax
00007FFDB3CD9146  jne     RtlAcquireSRWLockExclusive+02Bh (07FFDB3CD9152h)
00007FFDB3CD914C  mov     dword ptr [rsp+40h], 0FFFFFFFh
00007FFDB3CD9152  mov     rax, rdi
00007FFDB3CD9155  lock    cmpxchg qword ptr [rsi], rdx
00007FFDB3CD915A  jne     RtlAcquireSRWLockExclusive+04Ah (07FFDB3CD9174h)
00007FFDB3CD9160  mov     rcx, [rsp+70h]
00007FFDB3CD9164  call    RtlBackoff (07FFDB30DE28h)

```

```

00007FFDB30DE30A  je      RtlBackoff+70h (07FFDB30DE298h)
00007FFDB30DE30C  mov     ecx, dword ptr [rsp+8]
00007FFDB30DE310  mov     ecx, dword ptr [rsp+8]
00007FFDB30DE314  mov     ecx, dword ptr [rsp+8]
00007FFDB30DE318  pause
00007FFDB30DE31A  mov     ecx, dword ptr [rsp+8]
00007FFDB30DE31E  mov     ecx, ecx
00007FFDB30DE320  dec     dword ptr [rsp+8], ecx
00007FFDB30DE324  cmp     eax, eax
00007FFDB30DE326  ret
00007FFDB30DE328  jmp     RtlBackoff+54h (07FFDB30DE306h)

```

결과 및 분석

- 스레드 16개 비교

- 스레드 16개 일 때 SRLock > LockFree > SpinLock
- SRWLock과 busy-wait 동기화 방식의 차이 : 경합 발생 시 단위 시간당 lock 시행 횟수에 따른 명령어 부하
- SpinLock vs LockFree 차이 : 노드 생성 및 초기화 병렬 처리, interlock 명령어 부하 차이, interlock 명령어 시행 횟수 차이, interlock 병목 차이

감사합니다
